根据面试模糊记忆大致总结知识点，答案为 AI 给出，可能存在错误或不准确。仅用于参考学习

公司 A 逻辑验证工程师 一面/二面 offer

验证 vs 设计 的区别

简答：设计是**按 Spec 实现功能（RTL/PPA）**；验证是**证明实现正确（UVM/TB/覆盖率/断言/回归）**。

UVM 验证流程是怎样的？

简答：**读 Spec→提测试点→写 VPlan→搭 UVM 环境（seq/driver/monitor/scoreboard）→ref model/断言→随机激励+覆盖→回归收敛→交付**。

怎么交付（验证完成标准/交付物）

简答：功能&代码覆盖达标、断言全绿、回归通过；交付**报告+覆盖率+TB 代码+脚本+日志**，可复现。

我看你简历熟悉 DRAM，简单介绍一下

简答：DRAM 是**电容存储、需刷新、容量大/延迟高**的易失存储；**DDR** 是其高速接口标准，验证中常用内存模型模拟时序/延迟。

valid/ready 握手，先后顺序？

简答：**valid=数据有效、ready=可接收**；两者同高时一次传输成立，支持**背压/流控**。

NoC 的拓扑

简答：常见有 **Bus、Ring、2D Mesh、Tree/Fat-Tree、Crossbar、Torus/Clos**；高性能多核常用 **Mesh/Fat-Tree**。

NoC 的指标，怎么验证

简答：关注**带宽、延迟、吞吐、功耗、拥塞/利用率、乱序、丢包、死锁风险**；用 monitor 统计并做覆盖/断言。

做过芯片性能分析相关工作/研究吗？

简答：做过**并发场景压测、延迟统计、拥塞/背压观察**；虽非全量建模，但能基于 log/monitor 给出**性能诊断**与改进建议。

TOP 结构已定，实际性能达不到理论峰值，怎么分析？

简答：**先建理论模型（位宽×频率×效率）→再采集实测（时延/吞吐/利用率/背压）→差异归因（仲裁、buffer、路由、DRAM 瓶颈）→提出限流/加深缓冲/改仲裁等优化**。

UVM 部分八股：**fork…join**

简答：并发控制：**join（全等）/join_any（任一）/join_none（不等）**；常配合 **timeout+disable fork** 处理竞争/超时。

for 和 fork 一起用应该考虑什么问题

简答：**for=顺序循环，fork=并发线程**；for 里起 fork 要注意**局部变量捕获问题**（用中间变量绑定）。

robert robert robert robert

多核系统里是不是都用成熟方案？（你做了哪些非照搬）

简答：在 **UVM/AXI 等标准框架**上，自主做了**事务/约束定制、core_id/attr 扩展、多核并发/冲突场景、ref model/断言/覆盖设计**等。

RISC-V 和 x86 的区别

简答：**RISC-V = 开放 RISC、固定/可扩展指令、页表清晰（Sv39/48）；x86 = 封闭 CISC、变长指令、兼容包袱重**。

Pipeline **脚本的最大难点（通道类比）**

简答：**并发路由与仲裁、流量不均与背压传播、堵塞检测与释放、可控可复现的行为建模**最难。

Python 激励的难点

简答：**事务建模/约束依赖、并发调度/节奏控制、场景组合爆炸、与 SV 对接时序、失败追踪与可调试性**。

sequence 激励（怎么写/用）

简答：**sequence 生成 transaction，经 sequencer→driver 驱动 DUT**；常用**约束随机+定向 case**，并与**scoreboard/断言**联动。

要写哪些特殊激励

简答：**TLB hit/miss、跨页、大页、满容量替换；权限/异常（无 R/W/X、U/S 越权、无效 PTE）；非法 VA/未对齐；多核并发/乱序/背压压力场景**。

invalid 场景有哪些，异常信号如何定义？怎么做 scb 对比？

简答：如 **PTE V=0、R=0&W=1、X-only、非法层级/未对齐 VA/越页、U/S 权限冲突**；期望**触发正确异常并安全 Fail**。

AXI 会死锁吗？怎么解决？

简答：实现不当会**“假死锁/阻塞”（outstanding 过多、仲裁互等、ID/响应错配）；用**限窗/仲裁优化/及时响应、timeout 断言、背压观察、ID 配对与 credit 控制**化解。

robert robert robert robert

公司 B 芯片开发工程师 一面/二面 offer

验证流程？

A：从 **Spec→Testplan→UVM 平台（driver/monitor/scoreboard/ref model）→功能&代码覆盖→回归→调试收敛**，以覆盖率收敛为止。

覆盖率收敛：很多 bin 到不了，HP（高优先级）怎么收？

A：用 **工具标记不可达**（如 VCS/Verdi 报告、形式化不可达），**coverage-driven 激励+定向用例**补缺；**拆小 cross、分阶段收敛、优先级调度**。

AHB 怎么验证？（以 MMU 多为 Slave）

A：覆盖 **读/写/写回读、HTRANS/HSIZE/HBURST、HREADY 插入、HRESP=ERROR、未对齐** 等；用 **reg**

model/backdoor 做检查，monitor+scoreboard 比对，建 **协议覆盖**。

验证计划怎么做？

A：**Spec 提点→列 testpoint→映射 directed/random 用例→设计功能覆盖→设收敛指标（如寄存器 100%、HP 必命中、FC≥90%）**，用表格挂钩“测试→覆盖→预期”。

随机激励怎么实现？（constrained random）

A：定义 **sequence_item**（如 VA/attr/ASID/mode），加 **constraint/权重（dist/randc）** 限非法组合；结合 **coverage 反馈“缺啥补啥”** 动态引导。

既然有随机，为什么还要“排序/调度”工具？

A：**随机≠策略性覆盖**；用 **weighted/virtual sequence、testlist 优先级、sequence 仲裁** 先打基础场景，再攻低概率/HP 场景，显著加快收敛。

你为什么选择做“验证”？

A：验证对量产成败**权重极高**；MMU **状态/接口复杂**、能锻炼系统建模与调试深度，也反向提升我做 RTL/架构时的可验证性意识。

二面几乎没什么专业知识，综合面，只涉及项目

公司 C CPU 设计工程师 一面挂

1. 参数化序列检测器 撕代码，非常简单，输出检测个数，overflow 检测

问：序列检测怎么做？

答：用 FSM 或移位寄存器实现，FSM 表示匹配状态，移位寄存器则移入比特后与目标序列比较。用参数 SEQ_LEN 和 SEQ_PATTERN 控制移位寄存器和匹配模式，实现任意序列检测。

9. gem5 有哪些模型？gem5 提供了哪些 CPU/内存模型？

答：CPU：AtomicSimple、TimingSimple、Minor、O3；内存：Classic（简单）和 Ruby（支持一致性协议）。

10. gem5 框架是怎样的？

答：顶层 Python 配置，CPU 模型层，内存系统 (Classic/Ruby)，设备模型，底层是 C++ 事件驱动内核。

11. 为什么用 SE 模式？为什么选择 SE 模式？

答：SE 模式启动快、简单，适合指令级/微架构性能研究，不需要完整 OS。

12. 为什么用 FS 模式？为什么选择 FS 模式？

答：FS 模式能运行完整 OS，支持 I/O、多核一致性、TLB/page table 等系统级研究。

13. Ruby 内存系统

robert robert robert robert

问：Ruby 是什么？用过吗？

答：Ruby 是 gem5 的可配置内存系统，支持 cache coherence 协议（MESI/MOESI），可用于研究多核一致性。我调过一些开源配置，主要学习其 SLICC 协议描述和 cache 参数修改。

14. ACE vs CHI

问：你用的是 ACE 还是 CHI？

答：我用的是 CHI。相比 ACE 的总线 + snoop，CHI 基于分组化 NoC，支持大规模多核和目录协议，更适合 NPU/GPU/CPU 的一致性互连。

公司 D 芯片验证工程师 一面/二面 offer

Q：端到端数据流怎么在环境里完成比对？

A：driver 送入激励→DUT→input/output monitor 抓事务→参考模型生成期望→scoreboard 对 DUT 实际输出做 compare，覆盖整条数据路径。

Q：Cache 除了数据，还存哪些元信息？

A：典型有 tag、valid、dirty、LRU/替换位、权限/属性位以及一致性状态（MESI/MOESI），用来管理命中、写回与替换。

Q：monitor 抓到数据后能直接和期望比吗？

A：通常要先做格式/时序对齐与协议层抽象，再送入参考模型产出的 expected 与 actual 逐字段比较，避免假失败。

Q：一句话说清 AHB？

A：AMBA 的同步高性能片上总线，地址/数据分离流水、支持突发与（非 Lite）多主仲裁。

Q：AHB 读传输的关键时序是怎样的？

A：地址阶段给出 HADDR/HTRANS/HSIZE/HWRITE=0，下一拍进入数据阶段由 HRDATA 返回；HREADY 可插入等待，HRESP 给出结果。

Q：AHB 读通常消耗几个周期？

A：首个 beat 约 2 拍（地址+数据），突发后续每个 beat 理想 1 拍，等待状态会额外拉长。

Q：地址是谁发、数据谁回，还有别的信号吗？

A：Master 在地址阶段发控制（HADDR/HTRANS/HSIZE/HBURST/HWRITE），Slave 在数据阶段返 HRDATA/HRESP/HREADY。

robert robert robert robert

Q：Burst 下一个地址从哪一拍开始发？

A：地址与数据阶段流水并行，下一个地址在前一个数据阶段的同一拍就发出（非要等到“第三拍”是不对的）。

Q：“垃圾桶数据”在验证里指什么？

A：指无效/不可用的数据拍，如 HREADY=0 时的 HRDATA、错误响应或未填充完成的缓存行，需在监测/比对中筛除。

Q：什么是 trans（transaction）？

A：一次抽象的事务对象（一般继承 `uvm_sequence_item`），封装协议字段与数据，贯穿 sequencer/driver/monitor/scoreboard。

Q：driver、monitor、scoreboard 各自职责区别？

A：driver 驱动信号，monitor 被动采集封包为事务，scoreboard 功能比对（常配参考模型）。

Q：HTRANS 的含义？

A：2 位类型：00 IDLE、01 BUSY、10 NONSEQ（突发首拍）、11 SEQ（突发后续）。

Q：慢速 Slave 或被 Mask 的区域会不会堵住别人？

A：会造成当前 Master 阻塞；需靠等待→超时/错误响应→仲裁/分通道/QoS 等机制避免全局拥塞。

Q：多个 Master 通过 ACP 同端口访问会怎样？

A：ACP 是共享仲裁点，若当前事务占用且响应慢，其他 Master 会发生 Head-of-Line 阻塞，需靠 interleave/QoS/超时缓解。

Q：HRESP 表示什么？

A：AHB 响应：Lite 常见 OKAY/ERROR；全协议含 OKAY/ERROR/RETRY/SPLIT，配合 HREADY 在数据阶段有效。

Q：SPLIT 的作用是什么？

A：Slave 无法立即完成时返回 SPLIT 释放总线，稍后用 HSPLIT 唤回原 Master 继续，提升总线利用率。

Q：功能覆盖率你最关注哪些点？

A：协议关键字段取值、功能路径（读/写、hit/miss 等）、边界与异常、状态机状态/转移、以及关键组合交叉。

Q：功能点怎么系统划分来造用例？

A：从接口协议、功能路径、边界异常、状态机与 cross 组合五个维度拆分，再逐点设计定向/随机激励。

Q：scoreboard 的工作细节？

A：收输入事务→参考模型产 expected→收 DUT actual→按 ID/时间对齐匹配→字段 compare→记录 pass/fail

robert robert robert robert

与日志。

Q：UVM phase 是干嘛的？

A：统一生命周期：build/connect/elaboration/simulation 启动→run→extract/check/report/final，保证各组件有序协作。

Q：phase 的执行顺序记忆法？

A：build → connect → end_of_elaboration → start_of_simulation → run → extract → check → report → final。

Q：怎么判断“把这个用户/主机完全解除”？

A：其 outstanding 事务计数归零且相关 FSM 回到 IDLE，且没有新事务进入队列。

Q：Drop/Reset 这类废弃流程验证要点？

A：Drop 要保证事务释放与队列清空；Reset 需清零计数与复位状态机；平台结束前必须 outstanding=0 且 idle。

Q：UVM 里的 case 怎么跑？

A：命令行指定 `+UVM_TESTNAME=xxx` 或顶层 `run_test("xxx")`，回归用脚本批量调度 test×seed。

Q：virtual sequence 是什么场景用？

A：用于跨接口/多 agent 协同激励，统一调度多个普通 sequence 并行/顺序运行。

Q：sequencer 里面一般包含什么？

A：参数化事务类型、`seq_item_export` 端口、仲裁/调度逻辑、运行状态与（可选）配置/子 sequencer 引用。

Q：SystemVerilog 与 Verilog 的差异点（验证向）？

A：SV 是超集，新增 OOP、约束随机、功能覆盖率、接口、断言与高级类型，既能写 RTL 也擅长验证。

Q：SystemVerilog 做验证的优势？

A：OOP 可复用、constraint-random 高效探索、covergroup 量化闭环、interface 封装、SVA 断言与并发通信原语。

Q：SV/UVM 常用通讯手段有哪些？

A：TLM（seq_item/analysis）、mailbox（队列）、semaphore（互斥/限流）、event（同步），按场景取舍。

Q：scoreboard 与外部如何通讯？

A：monitor 用 `uvm_analysis_port` 广播事务，scoreboard 用 `analysis_imp/export` 接收，内部用队列/ID 匹配 compare。

robert robert robert robert

Q：覆盖率的基本概念？

A：衡量验证完整性；分工具统计的代码覆盖率与工程师定义的功能覆盖率，后者直接映射规格功能点。

Q：coverpoint（point）是什么？

A：功能覆盖率的基本单元，给变量定义 bins 统计命中；可加 `ignore_bins/illegal_bins` 管理统计范围。

Q：同一变量在不同条件下不同取值范围如何覆盖？

A：约束随机化限定范围，coverpoint 加 `iff(condition)` 条件化采样，再用 cross 覆盖条件×取值组合。

Q：功能覆盖率具体如何采集？

A：实例化 covergroup，在事务有效的时刻调用 `sample()` 或用事件触发；报告由仿真器导出再做闭环。

Q：`sample()` 的作用与用法？

A：触发一次覆盖采样（隐式事件或显式调用），应在有效且稳定的数据时机调用，避免过采样污染数据。

Q：要判断信号前两拍的状态怎么写？

A：SVA 用 `$past(sig,1)` 与 `$past(sig,2)` 取历史值，或配合 `$stable/$rose/$fell` 做时序关系检查。

Q：virtual sequencer 本体是什么、与 virtual sequence 配合怎样？

A：virtual sequencer 不直接连 driver，存放多个子 sequencer 句柄；virtual sequence 跑在其上以启动与编排多接口序列。

robert robert robert robert

公司 E 一面挂：

- 设计 vs 验证的区别是什么？

设计把规格变成可综合的 RTL/网表；验证用 UVM/断言/覆盖率等证明“按规格工作且边界可控”。

- 验证全流程长什么样？

规划 VPlan → 搭 UVM 环境 → 生成激励 → Scoreboard/断言检查 → 覆盖率收敛 → 回归与签核交付。

- “后仿真”指的是什么？

门级后仿真（GLS），用网表+SDF 回标验证时序/复位/X 传播，确保综合/布线后仍功能等价。

- CDC 核心问题与解法？

跨域致亚稳与位不一致，单比特用双触发同步，多比特用握手/异步 FIFO（Gray 指针），再配合 CDC 工具静态检查。

- AMBA 经历怎么更好地描述？

“AHB 写过 RTL、握手仲裁更熟；AXI/ AHB 在 IoT 用过 IP+VIP 做系统级验证与协议一致性检查。”

- AHB 的 response error 是什么？

Slave 用 HRESP=ERROR+HREADY=1 表示本次传输失败（如非法地址/权限），Master 必须终止当前传输。

□ “error”在 AHB 里的通用含义？

即一次总线传输未被成功完成的标准响应，需由上层软件/控制逻辑决定重试或异常处理。

□ HTRANS 有何作用？

2 位类型：IDLE/BUSY/NONSEQ/SEQ，NONSEQ 启动一次传输，SEQ 表示 burst 后续；HREADY=0 时必须保持稳定。

□ AXI 为什么要“五通道”？

AR/R 与 AW/W/B 完全解耦，读写并行、独立握手、支持多事务乱序与更高吞吐。

□ AXI “独占访问”（Exclusive）怎么保证原子性？

用 ARLOCK/AWLOCK 做独占对（读后写），全程监控 token，成功返回 EXOKAY，被他人干扰则写不生效。

□ UVM 的核心框架是什么？

Test 控全局，Env 管多个 Agent，Agent=Sequencer+Driver+Monitor；数据流“Sequence→Driver→DUT→Monitor→Scoreboard”。

□ UVM 寄存器模型（RAL）解决啥？

把 DUT 寄存器抽象成对象（block/reg/field），经 adapter 转成总线事务读写，并由 predictor 同步镜像实现自动化配置与一致性校验。

□ automatic task 的意义？

每次调用独立栈帧、可重入、适合并行环境（UVM 的 run_phase/body/drive 等任务基本都应 automatic）。

□ VCS 的“动作模型/实验装置”有哪些典型用法？

常见模式：RTL 仿真、GLS+SDF、UVM 回归+覆盖合并、断言/协议检查、UPF 低功耗、混合语言/PLI、并行加速（-jN/并发种子）。

□ “dump” 在仿真里做什么？

生成波形文件（VCD/VPD/FSDB）供 DVE/Verdi 回放；可按层级/时段/条件开启以控体积与性能。

□ “atomic” 的通用含义？

不可分割的操作语义：要么全部完成要么不发生；AXI 用独占访问实现，软件用 std::atomic/CAS，验证用信号量临界区。

□ SystemVerilog 里“几种 task 类型”？

按执行：static vs automatic；按归属：module task / class task / virtual task；在 UVM 中几乎一律 automatic，需多态的再 virtual。

公司 F AI 芯片研发工程师 一面挂

robert robert robert robert

回归 / 覆盖率 / Flow

1. **Q：你们的 regression flow 是怎样的？**
   A：Jenkins/脚本触发 → 并行调度仿真（VCS/Questa/Xcelium）→ 解析日志（UVM_ERROR/FATAL、seed）→ 合并 coverage（URG/IMC）→ 生成 HTML/CSV 报告并邮件通知。

2. **Q：为什么做大规模随机回归后还要看单个 case 的覆盖率？**
   A：为分析**覆盖贡献度**与**冗余度**、快速定位未命中 coverpoint，从而删冗增效、指导约束/权重调整与 directed case 投入。

3. **Q：发现不同配置/用例对覆盖贡献差异后，你会做什么？**
   A：调约束权重（biased random）、补 directed/semi-directed、按 config 贡献重分配回归资源、构建最小回归集并回写 testplan 追踪 closure。

4. **Q：回归与覆盖率的关系？**
   A：回归是执行与收敛过程，覆盖率是质量度量；以**功能覆盖率**（covergroup）为主闭环，**代码覆盖率**用于发现死角与冗余。

robert

随机激励 / Python vs SV

5. **Q：你的 Python 随机是纯随机还是受约束的随机？**
   A：做**受约束/加权随机**（constraint/weighted），保持多样性的同时对低命中场景提升权重以加速 coverage 收敛。

robert

6. **Q：Python 的随机能力 vs SV 的随机约束？**
   A：两者定位不同：Python 更灵活、易整合与反馈闭环；SV constraint 与仿真环境无缝集成、体系化强；工程上常**结合使用**。

robert

7. **Q：Python 受约束随机常用库？**
   A：基础分布用 random/numpy.random，复杂约束用 python-constraint/z3-solver，配置与反馈用 yaml/json+pandas。

robert

8. **Q：在 Python 里如何构造“像 SV 约束那样”的随机？**
   A：用“条件过滤+概率分布”或借助求解器（如 z3）生成满足约束的取值，再将 transaction 推送至 SV 环境。

Python↔SV 交互 / 文件 vs 内存

9. **Q：为什么用“文件交互”而不是内存通信？**
   A：文件 I/O **最简单稳定、工具无关、可复现**（留存刺激输入），适合项目初/中期；但高频交互/SoC 级会受 I/O 开销与实时性限制。

10. **Q：大规模/SoC 级时如何提效？需要包什么 wrapper？**
    A：用 **DPI-C 的 C wrapper** 做内存级交互（SV↔C↔Python），或用 socket/ZeroMQ/gRPC 跨进程通信；模块级可文件，IP/SoC 级建议 DPI，系统级可 RPC。

11. **Q：你有 DPI 经验吗，基本链路是？**

A：了解并能写基础例子：SV import "DPI-C" 调 C，C 再调 Python C-API；DPI 适合同步/高速、文件不适合的场景。

参考模型 / Scoreboard / Monitor / Checker

12. **Q：SV 更偏“控制”，那你的 Reference Model 是什么、怎么接？**

A：RM 负责**结果预测**（功能算法），与 DUT 同吃输入、在 scoreboard 做逐字段对比；SV 侧负责驱动/同步/调度与连接 RM 输出。

13. **Q：乱序匹配在 scoreboard 里怎么做？**

A：维护 DUT 队列与 RM 队列，用 **transaction ID/tag** 匹配，再 field-by-field 比较；容忍窗口/阈值用于时延/浮点等容差。

14. **Q：你的 DVT（对比数据）来源是顶层还是中间结构？**

A：**以顶层接口事务为主**做正式比对（黑盒、可复用），关键中间点只加 debug/coverage monitor，不直接参与最终判定。

15. **Q：两条传输线比较需不需要各自维护对应关系？**

A：若严格保序的单通道可直接一一比；但存在并行/乱序/跨通道组装时，**匹配应上提到 scoreboard/comparer 统一维护**。

16. **Q：Checker 是组件还是概念？是碎片 if 还是体系？**

A：两者皆是：概念上是“规则判定”，实现上**可做独立组件/断言集**；从零散 if 演化为**可复用的 protocol/behavior checker**。

17. **Q：为什么不是 monitor 来做时序检查？monitor 与 checker 的边界？**

A：**monitor 只采集还原事实（sample→transaction）**；**checker 依据规则判定正误（时序/协议）**；driver 只负责驱动，不判 DUT 响应对错。

18. **Q：Reference Model 要集成到上层（不再用 Python 文本驱动）时如何工作？**

A：把 RM 改为**监听模式**：上游接口 monitor 产出 transaction → 通过 analysis port/DPI 喂给 RM → RM 产出期望 → scoreboard 对比。

19. **Q：compare 是什么？**

A：scoreboard 的**字段级对比单元，应该是 UVM 自带功能**：匹配到同一事务后逐字段比较、打印差异、计数统计，并支持阈值/宽容策略与乱序匹配。

20. **Q：参考模型需要做到 cycle-accurate 吗？**

A：面向功能验证通常**功能级精度**即可（可加时延容忍窗），若做 pipeline/性能验证再演进为 cycle-accurate。

21. **Q：仅有功能级就能完成 Transaction-Level 对比吗？**

A：能；Transaction 层只看**输入→输出的逻辑映射**，不要求逐拍对齐，速度快、对 RTL 结构变更不敏

robert robert robert robert

感。

22. **Q：握手信号抢拍/中断导致未完全回收，该怎么监控？环境有 driver/monitor 吗？**
A：有 driver/monitor；**时序违规**由 protocol/temporal checker（或 SVA）监控，monitor 提供事实，driver 只负责合法驱动。

UVM Phase / 执行顺序

23. **Q：UVM 的 phase 是什么？**
A：是**统一生命周期调度**：build/connect → run（task+objection）→ extract/check/report/final，保证组件同步有序。

24. **Q：build phase 组件执行顺序？**
A：**Top-down/Depth-first**：父先执行自身 build，再 create() 子组件递归构建；保证 config/factory 在子组件创建前就位。

25. **Q：run 之后的 Future Phases（extract→check→report→final）顺序怎样？**
A：各阶段按 **Bottom-up**（自下而上）调度组件，先子后父，便于自底向上汇总统计/检查与生成报告。

UVM Factory / Override / 宏注册

26. **Q：UVM 工厂的 type/instance override 需要哪些前提条件？**
A：类需继承 uvm_object/component、用 uvm_*_utils 注册、通过 type_id::create() 创建，且**在创建前**设置 override（实例 override 还需路径匹配）。

27. **Q：注册宏有什么效应？**
A：把类型注册进工厂、生成 type_id/get_type() 等反射接口，使 **factory 可识别/创建/override** 此类；无宏注册就无法被工厂使用。

Sequencer / 虚拟激励

28. **Q：PSequencer 与 MSequencer 的关系？**
A：**PSequencer（虚拟/父）用于协调多个 MSequencer**（各 agent 本地 sequencer）；virtual sequence 通过 p_sequencer 同时驱动多接口。

AMBA / AXI 核心概念

29. **Q：AXI 的 outstanding 与 interleaving 是什么？**
A：**Outstanding**：未完成事务可并存（主机连续发 AR/AW）；**Interleaving**：返回数据可交错（以 RID/BID 区分）。

30. **Q：Outstanding 与乱序（out-of-order）是什么关系？**
A：**Outstanding 是乱序的前提**：无并存就无乱序；同 ID 保序、不同 ID 可乱序由从设备调度。

手撕代码：

1：写了一个约束，超级简单，10 行就写完了

robert robert robert robert

2：写了一个握手信号的 UVM Driver，也很简单

公司 G 芯片验证工程师 一面/二面 offer

Q: 功能覆盖率和代码覆盖率有什么区别?

A: 代码覆盖率关注 RTL 是否被执行（行、分支、表达式、toggle）；功能覆盖率关注 Spec 中的功能点是否被测试（covergroup/coverpoint/cross）。两者配合评估验证充分性。

Q: 为什么在验证中使用 Python 生成激励?

A: 处理复杂数据/表项（如 MMU 页表、地址映射）更高效，可生成 CSV/二进制供 UVM 读取，降低 SV 约束复杂度、提升可控性和复用性。此思路通常由我主动提出并落地。

Q: 在小团队里你会主动验证哪些内容?

A: 接口协议时序、核心功能流程、异常/边界场景（错误响应、复位、背压）、以及性能相关的突发与并发；并用覆盖率闭环验证计划。

Q: 你如何验证 AHB 接口?

A: UVM 中用 driver 发起 SINGLE/INCR 等突发读写，monitor 采集事务，scoreboard 做数据一致性比对，结合 VIP 的协议 checker 与自定义覆盖率覆盖 HTRANS/HBURST/HSIZE/HRESP 组合。

Q: RTL 常用的 AHB 传输类型及验证要点?

A: HTRANS 以 NONSEQ/SEQ 为主（同时验证 IDLE/BUSY 忽略性），HBURST 常用 SINGLE/INCR（也测 INCR4/8 与 wrap 地址），HSIZE 多为 word 同时覆盖 byte/halfword 与未对齐访问。

Q: Always-on（AO）域如何验证?

A: 关注掉电/上电时序、跨电源域隔离、防 X 传播、状态保持与唤醒路径；UVM 激励+SVA 检查时钟/复位稳定，配合 UPF/CPF 静态一致性检查。

Q: 你用过 AHB/AXI 的 VIP 吗，怎么用?

A: 用作 master/slave 代理驱动事务，内建协议 checker 与覆盖率，monitor 输出 transaction 供 scoreboard 与参考模型比对，快速搭建且保证协议合规。

Q: VIP 是只做总线协议验证还是集成到整体验证?

A: 两者都做；早期做协议核查，系统级把 VIP 作为 UVM agent 集成，驱动/采集/覆盖/比对一体化，实现从接口到功能的闭环。

Q: SystemVerilog 中 function void swap(int a,b)能否交换实参?

A: 不能，默认值传递；需改为 ref 参数（或 output/inout）才会交换外部变量。

Q: rand 与 randc 的区别?

A: rand 独立随机可重复；randc 在取值空间内循环遍历不重复，适合需要覆盖全部取值的场景（如 opcode/burst_len）。

robert robert robert robert

Q: 两个对象实例各自含同名 randc 变量，会互相影响吗？

A: 不会；randc 循环状态按实例独立维护，实例间互不干扰。

Q: 约束很多、求解变慢时如何提效？

A: 拆分约束、减少跨变量耦合、将可计算逻辑移至 post_randomize()、合理缩放解空间、使用 randc/dist、分层随机化、借助 solver 日志定位冲突。

Q: 约束拆分与 post_randomize() 的作用？

A: 拆分降低求解复杂度；post_randomize()把可计算/修正类逻辑从求解器移出（如对齐/映射），显著减轻 solver 负担。

Q: soft 约束的作用？

A: 提供“默认”弱约束，被更强的 with 约束或更具体的类内约束自动覆盖而不冲突，便于在通用环境中给出默认范围又保持场景灵活性。

Q: UVM sequence 常用的启动方式？

A: seq.start(sequencer)为主；可用 uvm_do/uvm_do_on 宏简化；在 virtual sequence 里启动多个子序列实现多 agent 协同与并发。

Q: UVM 工厂机制是什么、为什么用？

A: 通过::type_id::create()统一创建对象/组件，并用 type/instance override 在不改源码下替换实现，增强环境复用与可扩展性。

Q: 工厂覆盖（override）有哪些？

A: Type override（全局替换某类的所有实例）与 Instance override（仅替换指定路径组件），分别用 set_type_override 与 set_inst_override。

Q: UVM 环境如何连接 DUT？

A: 用 SystemVerilog interface 封装 DUT 端口，tb_top 实例化并连 DUT；通过 uvm_config_db 传递 virtual interface 给 driver/monitor，驱动与采集信号。

Q: 传递 virtual interface 的常用方式？

A: 在 tb_top 用 uvm_config_db::set 设置，在组件 build_phase 里用 get 获取；这种方式解耦顶层与组件，便于复用和层级化管理。

Q: 用 Python 生成激励与 UVM 配合的典型落地方式？

A: Python 离线生成 CSV/二进制或 SV 可读 mem 文件→UVM sequence 读取加载→driver 驱动事务→monitor 采集→scoreboard 与参考模型比对→覆盖率闭环。

Q: 在 AHB 验证中常见异常/边界场景有哪些？

A: 未对齐访问、非法地址返回 ERROR、wait-state 插入（HREADY 低）、burst 跨边界与 wrap、复位中断传输、背压导致的 FIFO 溢出/欠读。

Q: 如何确保覆盖率闭环？

robert robert robert robert

A：根据验证计划建立 covergroup（协议字段与功能状态/异常交叉）、与代码覆盖率联合评估；对未命中项补 case 或调整约束/场景直到收敛。

公司 H 芯片验证工程师 一面挂

（我真不理解为啥这么喜欢问形式验证，我都没做过还问）

验证需要的要求是什么？

功能正确、覆盖闭环（功能/代码）、环境可复用与可配置、结果可追踪并能自动回归。

（性格）你有哪些性格优势？

有责任感、逻辑强执行快、抗压稳定；能把问题闭环到位，尤其适合高强度回归阶段。

验证和设计的区别？

设计“实现功能”，验证“证明正确”；前者产出 RTL，后者产出 UVM 环境/测试/覆盖与结论。

什么是形式验证（Formal/形式）？

基于数学推理的验证，不靠激励而靠断言与求解器穷举，适合控制逻辑与小规模模块。

什么是 STA？

静态时序分析；不跑激励，只计算组合路径与时钟关系，检查 setup/hold 与时序裕量是否满足。

formal 在 MMU 里怎么用？

对 TLB 替换/一致性、页表遍历 FSM 死锁、异常互斥、复位清零等关键控制点做属性证明与覆盖可达性。

一个“验证链/闭环”都包含哪些部分？

环境搭建→激励生成→结果比对→覆盖分析→回归与报表；形成需求到结论的可追溯闭环。

formal 包含哪几部分？

Assume（环境假设）、Assert（待证性质）、Cover（可达性/覆盖）、Constraints（输入范围/时钟复位）、Solver（求解器）。

什么是功能验证（Functional）？

用 UVM 环境+随机/定向激励驱动 DUT，与参考模型/断言比对，证明行为符合规格。

形式验证的功能/常见 App？

属性检查、等价检查、连线/复位/CDC/X 传播、安全与覆盖可达；对应 Jasper/VCF/Questa 的 Property/LEC/Connectivity/CDC/Reset/Xprop 等 App。

为什么 formal 验证需要验证“不定态（X）”？

保留 X 能暴露未复位、竞争冒险、case 遗漏和 CDC 缺陷；强行清零会“假通过”。

验证里“输入”是什么？

驱动到 DUT 的激励（数据/控制/协议信号）；在 UVM 由 sequence→driver 产生，在 Formal 用 assume 约束为合

robert robert robert robert

法集合。

为什么还要做动态仿真？

只有动态仿真能覆盖复杂系统交互、长数据通路与异常流程；STA/Formal 无法替代系统级场景验证。

为什么要做后仿（门级仿真）？

验证综合/布线后的网表在真实延时下是否仍正确，重点查时序、复位、X 传播与 DFT 插入影响。

MMU 为什么要存在？

提供 VA→PA 映射、内存保护与隔离，支撑多任务与虚拟内存管理，并通过 TLB 加速访问。

CPU 不能直接拿物理地址吗？

用户态程序通常拿不到 PA；在开启 MMU 的系统中 CPU 只见 VA，PA 由 MMU 翻译，内核或验证环境才能解析/比对 PA。

如果关闭 MMU 能直接用物理地址吗？

能；如 Boot 阶段或裸机/仿真可 VA=PA 直访，但会失去隔离与保护，不适合多任务 OS。

robert robert robert robert

公司 | 芯片验证工程师 一/二面 Offer

UVM 的 agent 里都有哪些组件？它们之间数据怎么流？

agent 通常包含 sequencer、driver、monitor。sequence 产生 transaction → sequencer 下发 → driver 转换为接口激励；monitor 从接口采样还原为 transaction，送到 scoreboard/coverage。

自研 agent（非第三方 VIP）需要自己实现什么？

需要自定 transaction 类型、driver 时序激励、monitor 采样/协议解析、TLM 连接与分析端口，还要依据协议补齐异常路径与断言。这样能精确贴合项目定制需求。

driver 是怎么“驱动”DUT 的？

driver 在 run_phase 里用 get_next_item(req) 取到 transaction，按协议把字段映射到 virtual interface 信号并完成握手，最后 item_done()。例如 AHB 要正确处理 hready/htrans/haddr/hwdata 等时序。

DUT 有响应时，driver/环境如何把响应“传回去”？

把读回数据/响应码等封装为 response，通过 put_response(rsp) 回给 sequencer 或上层。关键携带字段包括 txn_id/tag、数据、状态码以及必要的时序统计信息。

多并发/乱序场景下 Request 和 Response 如何一一匹配？

为每个请求分配唯一 ID/tag；发出时记录到 pending 表，响应返回时按 ID 查找并关联。AXI 用 *ID*，自研 MMU 可用 *req_tag*；scoreboard 用关联数组完成匹配与清理。

创建 response 时直接 new() 会丢信息吗？如何避免？

会。直接 new() 会丢失原 request 的 ID/上下文字段；应使用 clone()/copy() 继承关键字段，再补上响应数据与状态，确保后续匹配与比对正确。

黑盒测试与白盒测试有什么区别？

黑盒只看接口行为、依据规格；白盒可观察/断言内部状态与路径。实际项目多用黑盒为主、白盒为辅（关键状态机/寄存器加断言和 coverpoint）来补充结构覆盖。

“特异点/异常”应从哪些维度拆解为测试点？

从功能（读写/页表层级/权限）、时序（back-pressure/乱序）、结构（多模块交互/流水级）、数据模式（边界/跨页/对齐）、以及错误处理（非法 PTE/权限错/超时）五个维度系统拆解。

常见的测试用例设计方法有哪些？

边界测试（上下限/跨页/计数溢出）、等价类划分（各类 burst/页大小各取代表）、随机约束（UVM CRV 覆盖大空间）、覆盖导向（按 coverage 缺口补 directed）、错误注入（非法/时序异常）、交叉组合（页大小×权限×乱序）。

MMU 模块从请求到响应的整体流程是怎样的？

请求进入→TLB 查找命中则直出→未命中触发 Page Table Walk 逐级解析 PTE→成功得到 PA 并填回 TLB，异常则返回错误。验证上用 reference model 复现页表遍历，与 DUT 输出在 scoreboard 比对。

monitor 与 scoreboard/参考模型之间交换什么信息？

monitor 以 transaction 形式通过 analysis_port 输出实际观测到的类型、地址/数据、响应码、事务 ID 等。scoreboard 用参考模型生成 expected，与 actual 做字段级比较并记录覆盖。

覆盖率没到怎么办（测试点与 coverage 的对应）？

先定义功能/交叉覆盖映射到需求与测试点；回看 coverage 报告找 gap，用定向 directed case 或定向约束补齐；必要时增加白盒 coverpoint/断言提升可观测性。

UVM 中 sequencer/driver/monitor 的 TLM 连接要点是什么？

sequencer 与 driver 通过 seq_item_port/export 进行 request/response 交互；monitor 通过 analysis_port 广播到 scoreboard/coverage；组件在 connect_phase 统一连通。

在 driver 层如何保证协议严谨性？

按 ready/valid/握手次序发波形，非法转移加断言；对 back-pressure、等待周期、突发边界等建立 helper 任务，确保波形稳定且可复用。

乱序返回下 scoreboard 如何保证正确性与线程安全？

以 ID 为键的关联表 + 原子更新；写比较前确保 request 已入表，比较后删除并记录覆盖；必要时加入 mailbox/queue 的仲裁或轻量锁，避免竞态。

错误注入（MMU/AHB）应该怎么做？

MMU：造 invalid/permission 错误 PTE、跨页越界、未对齐；AHB：hresp 错误、延迟/超时、非法 htrans。验证 DUT 能否正确检测、上报并恢复。

边界场景的典型列举（便于快速答题）？

地址对齐/页边界±1、计数器溢出/回绕、突发长度极值、TLB 满/空、最小/最大页大小、最短/最长 back-pressure、最小/最大 outstanding。

robert robert robert robert

自研 agent 与第三方 VIP 的取舍如何表述？

自研灵活、可定制、易插入私有字段与调试点；VIP 成熟省时但黑盒度高、定制受限。项目若协议标准且时间紧可用 VIP，若强定制或新协议更偏自研。

其他的没写的就是没怎么问八股，总体上喜欢问一堆八股的公司并不是很多，大部分还是习惯拷打项目，项目这个东西因人而异，但是必须要做到对项目的技术细节或至少是自己负责的技术细节十分熟悉，并能够完整的说出底层原理。

robert robert robert robert